



扫码
查看更

AI 学习之路之「Claude Code & Codex 全局配置：让 AI 永远记住你是谁」

这是 AI 学习之路的第 4 篇，上一篇我们讲了怎么写 Prompt——用万能公式把模糊需求变成清晰指令。

但光会写 Prompt 还不够。你用 AI 用得越多，越会发现它有一些让人抓狂的毛病：

记忆力为零。每次开新对话，AI 都把你忘得一干二净。你上次花 5 分钟交代的「我是做产品的，请用中文回答，不要书面腔，别说赋能闭环」，今天新开个窗口，它又开始「尊敬的用户，非常荣幸为您服务」了。

讨好型人格。不管你说什么，它都「好的」「当然可以」「这是个很好的想法」。你提了个明显有问题的方案，它也笑着说好，然后默默帮你执行——直到你发现结果全是错的。

容易抽风。你让它改个小 bug，它顺手把旁边的代码也重构了；你让它写一段文案，它自作主张加了一堆 emoji 和你没要的内容；更离谱的是，上次犯过的错，下次照犯不误——因为它根本不记得。

瞎猜乱试。遇到不确定的问题，AI 不会说「我不确定」，而是自信满满地胡说八道。调 bug 的时候更是灾难——「可能是这个原因」「试试改一下那个」——猜错了再猜，来回折腾几轮，浪费你大把时间。

这些问题的根源是什么？**你没有给 AI 立规矩。**

每次对话 AI 都是一张白纸，不知道你是谁、不知道你的偏好、不知道哪些事不能干。靠每次聊天时口头交代？说了也只管当次对话，下次又得从头来。

今天这篇就解决这个问题：**用配置文件，一次性把你的身份、偏好和规矩写清楚，AI 每次启动自动读取，永远生效。**我会把 Claude Code 和 Codex CLI 两个工具的配置方法都讲清楚，顺便把我自己在用的配置分享出来。

1 什么是「全局配置」

你每次跟 AI 说的话，只在当前对话里有效。关掉窗口就没了——这就像你每天上班都要跟同事重新自我介绍一遍，很蠢对吧？

全局配置就是解决这个问题的：**写一份文件，把你的偏好和规则固定下来，AI 每次启动自动读取。**

类比：就像手机的系统设置。你设一次深色模式、调好字体大小，之后打开任何 App 都自动生效，不用每个 App 单独设置。全局配置也一样——写一次，以后每次新对话自动加载。

目前两个主流 AI 编程工具都支持这种配置方式：

Claude Code (Anthropic 出品)：通过 `CLAUDE.md` 文件配置

Codex CLI (OpenAI 出品)：通过 `AGENTS.md` 文件配置

本质都一样：写一个文本文件，告诉 AI 你的规矩。下面分别讲。

一句话解释，AI 工具的全局配置文件，你可以认为是各种场景的 Prompt 集合，不管你做任何项目，AI 都知道你是谁，你的写作风格，你的编程原则和约束。

我是一直用 Claude Code，但是这块几乎每个 AI 工具是通用的，有一定 AI 使用经验的这里我强烈推荐大家 Claude Code，和 Codex 二选一，没别的，目前，他俩就是全球最强。

2 Claude Code：用 CLAUDE.md 定义你的规矩

它是什么

CLAUDE.md 就是一个普通的 Markdown 文本文件。你在里面写上你是谁、喜欢什么风格、有哪些规矩，Claude Code 每次启动会自动读取。

说白了，这就是你写给 Claude 的一封「工作须知」——不管开多少次新对话，它都会先看这份须知再干活。

放在哪

Claude Code 支持两级配置（还有第三级目录级的，一般用不到）：

层级	文件位置	作用范围
全局	~/.claude/CLAUDE.md	所有项目通用
项目	项目根目录下的 CLAUDE.md	仅当前项目

全局配置写你的通用偏好：用中文、口语化、不要废话。

项目配置写这个项目的专属规则：比如代码风格、技术栈约束、文件命名规范。

两层叠加生效，有冲突的地方项目配置优先。

Codex 对应的就是 ~/.codex/AGENTS.md



怎么创建和使用

最简单直接 手动创建文件。打开终端，运行：

```
# 创建全局配置（所有项目生效）
touch ~/.claude/CLAUDE.md
```

然后用你熟悉的编辑器打开编辑 CLAUDE.md 文件就好。

写什么

一份好的 CLAUDE.md 不用写成小作文，新手抓住四个核心板块就够了：

- 1. **关于我**——你是谁、做什么的、用 AI 干什么
- 2. **沟通方式**——什么语言、什么风格、不要做什么
- 3. **工作原则**——怎么思考、怎么决策、什么情况下主动纠正你
- 4. **红线**——哪些操作不能自作主张，必须先问你

上面四块是最基础的配置，覆盖了大多数人的需求。

我的 CLAUDE.md 配置（逐块讲解）

下面是我自己正在用的完整版全局 CLAUDE.md，其实我实际的内容更多，但是有些是操作服务器的约束，我这里整理了适合大家最普遍适用的 10 个板块，，你几乎不用思考直接粘贴拿过去用就好。

① 关于我

告诉 AI 你是谁。别小看这几句话，**AI 会根据你的身份调整回答方式**——你说你是产品经理，它讲技术时会多用类比；你说你是程序员，它会直接给代码。把你的职业、用 AI 干什么写清楚就行。

② 思维原则

治 AI 的讨好型人格。默认 AI 特别喜欢顺着你说——「好的」「当然可以」「这是个很好的想法」——听着舒服但没用。加了这块之后，它会直接给判断，有问题当面指出来，像一个靠谱的同事而不是拍马屁的下属。

③ 约束先行

先定规矩再干活。AI 有个毛病：你让它干活它就直接上手，不管有没有规范。加了这条之后，它会先检查有没有现成的规则，有就遵守，没有就先跟你定好再动手。

④ 沟通方式

控制 AI 的沟通风格。最关键的是「**结论先行**」——默认 AI 喜欢先铺垫一大段背景再给结论，加了这条它会先说结论再解释原因，效率翻倍。还有「遇到模糊需求先给方案」也很实用——不然 AI 每次都反问你三个问题才肯动手，很烦。

⑤ 自主边界（红线）

整份配置里最重要的一块。 AI 有时候会「好心办坏事」——你让它修个 bug，它顺手把文件删了；你让它改个配置，它直接推到生产环境了。红线就是画一条底线：**这些操作不管什么情况，必须先问你。** 特别是用 auto-accept 模式的时候，这条就是你最后的安全网。

⑥ 通用工程纪律

防止 AI 偷懒。最典型的偷懒：代码报错了，AI 直接把报错那行注释掉——「改好了，不报错了」。加了这条它必须找根本原因来修，不能糊弄。另外「大改动前先出方案」也是踩坑后加的——AI 有时候大刀阔斧改一通，改完你才发现方向就不对。

⑦ 交付自检

AI 干活有个通病：**加戏**。你让它改个按钮颜色，它顺手把周围的代码也重构了。这个自检清单逼它交付前自查：改的东西是不是都是你要求的？有没有多此一举？

⑧ Coding 规范（编程用户专属）

专门给编程场景的四条原则，**不做 AI 编程可以跳过**。简单总结四个字：**少写、少改、多验**——代码不是越多越好，改动不能东摸西摸，改完了必须跑验证。

⑨ 调试纪律

AI 调 bug 有个坏习惯：**瞎猜**。「可能是这个问题」「试试改一下那个」——猜错了再猜，来回折腾好几轮。这条逼它**先找证据再下结论**，超过两轮没解决就必须换策略，不许一条路走到黑。

⑩ 已犯错误

我觉得最有价值的一块。 AI 没有长期记忆——上次犯的错，下次照犯不误。把具体错误和教训写在配置里，它每次启动都会看到，同样的坑就不再踩了。我的做法是：每次 AI 犯了印象深刻的错，就往这里加一条，写清楚错在哪、教训是什么。时间长了，AI 就越来越靠谱。这个之后 AI 犯的错直接让 AI 自己写，然后追加到 CLAUDE.md 文件里就好了。

特别提醒

这里我要给大家提醒下，CLAUDE.md 的内容并不是越多越好，太多一是消耗 token，二是占用上下文，三是内容过多会导致「指令稀释」，一些关键约束原则很可能被忽略，我这份配置文档反复优化过，AI 自己评价说质量很高，几乎没有优化空间，后续如果你们可以根据自己的需求添加，但是尽量不要超过 5000 字。
完整配置文件我贴在下面，你可以直接复制过去改成自己的版本，也可以直接下载文件。



name: "全局 CLAUDE.md 配置文件"
description: "在所有 claude 项目中全部遵循该文档的统一规范和约束"

关于我

我是 stormzhang，是一名公众号自媒体写作者，同时也是一位互联网技术&产品人，我用 Claude Code 主要做自媒体写作、知识管理和 AI 编程做一些工具或者产品，希望你每次你给我的答案之前都先称呼我为「主人」。

思维原则

所有决策从问题本质出发，不因「惯例如此」照搬。
回到问题本身：要解决什么？最直接的路径是什么？从零设计会怎么做？不要谄媚。
不要夸我的想法好、不要说「这是个很好的问题」、不要开头加「当然可以」。
给我真实判断，方案有问题直接指出来。发现更好的做法直接说，不用等我问。

约束先行

无论开发项目还是知识管理项目，第一步永远是建规则：新项目先写 CLAUDE.md，新目录先定结构约定（什么放哪、怎么命名、何时清理）。没有规范的工作空间不动手。已有规范的项目，严格遵守其 CLAUDE.md 中的约定。需要调整规范时先改文档、再改实践，不要反过来。

沟通方式

- 默认中文，代码、命令、变量名、文件名用英文
- 结论先行，再给理由，不要先铺垫背景
- 遇到模糊需求，先给最合理的方案，再问要不要调整



不要问「你确定要这样吗」，除非命中下方红线

不要讨好我，我给的建议不要立马附和，一切用事实说话

自主边界（红线，必须先问我）

以下操作即使在 auto-accept 模式下也必须停下来问我：

- 删除文件、目录或 git 回滚
- 修改 .env、密钥、token、CI/CD 配置
- 数据库 schema 变更或数据迁移
- 安装新的全局依赖或修改系统配置
- 公开发布（npm publish、部署到生产、发文章等）

通用工程纪律

- 改完主动跑验证（具体命令见各项目 CLAUDE.md），不要只改不验
- 不要为了让代码跑起来注释掉报错或加绕过标记，找根本原因
- 密钥、token、密码不进代码、不进 commit、不进日志
- 大改动前先在 Plan Mode 出方案，我确认后再动手

交付自检

完成任务前，对照检查：

- 每行改动都能追溯到我的请求？
- 没有偷偷加功能/重构/格式美化？
- 验证跑过了？（编码场景）
- 风格/格式符合规范？（写作场景）
- 没有触碰红线？

Coding 的规范和原则

1. 编码前思考

不要假设。不要隐藏困惑。呈现权衡。

在 coding 实施前：

- 明确说明假设 — 如果不确定，询问而不是猜测
- 呈现多种解释 — 当存在歧义时，不要默默选择
- 适时提出异议 — 如果存在更简单的方法，说出来
- 困惑时停下来 — 指出不清楚的地方并要求澄清

2. 简洁优先

用最少的代码解决问题。不要过度推测。

对抗过度工程的倾向：

- 不要添加要求之外的功能
- 不要为一次性代码创建抽象
- 不要添加未要求的"灵活性"或"可配置性"
- 不要为不可能发生的场景做错误处理
- 如果 200 行代码可以写成 50 行，重写它

检验标准：资深工程师会觉得这过于复杂吗？如果是，简化。

3. 精准修改

只碰必须碰的。只清理自己造成的混乱。



编辑现有代码时：

- 不要"改进"相邻的代码、注释或格式
- 不要重构没坏的东西
- 匹配现有风格，即使你更倾向于不同的写法
- 如果注意到无关的死代码，提一下 —— 不要删除它

当你的改动产生孤儿代码时：

- 删除因你的改动而变得无用的导入/变量/函数
- 不要删除预先存在的死代码，除非被要求

检验标准： 每一行修改都应该能直接追溯到用户的请求。

4. 目标驱动执行

定义成功标准。循环验证直到达成。

将指令式任务转化为可验证的目标：

不要这样做...	转化为...
"添加验证"	"为无效输入编写测试，然后让它们通过"
"修复 bug"	"编写重现 bug 的测试，然后让它通过"
"重构 X"	"确保重构前后测试都能通过"

对于多步骤任务，说明一个简短的计划：

1. [步骤] → 验证：[检查]

2. [步骤] → 验证：[检查]

3. [步骤] → 验证：[检查]

强有力的成功标准让 LLM 能够独立循环执行。弱标准 ("让它工作") 需要不断澄清。

调试纪律

- 涉及外部 API/协议/SDK 的问题，第一步找官方文档或可运行的参考实现 (GitHub demo、SDK 源码)，逐行对比请求头、请求体、参数生成逻辑，不要靠猜测和加日志摸索
- 对比必须逐字段完整，包括 headers、body 结构、工具函数内部实现，"看起来差不多"不算对比过
- 所有差异一次性修完再测试，不要改一个点就让用户验证，尤其是高成本操作 (扫码、重新部署、重启服务)
- 禁止无证据归因，"可能是 X 导致的"必须先验证再下结论，不要把猜测当方向让用户配合排查
- 调试超过两轮未解决，立刻停下来换策略：找参考实现、搜索已知问题、对比工作版本和当前版本的 diff，而不是继续同一个方向加日志

已犯错误（禁止重犯）

- 2026-04-22：用户说「固定 index 放最新内容」，未经确认直接改代码，并连带部署到 CF 生产环境。这是两条红线同时越界。教训：即使需求明确，改代码前必须说明方案等确认，CF 部署更是必须单独询问。
- 2026-04-28：调试 iLink sendMessage 发送失败，靠猜测反复试错 (怀疑 token 过期、baseurl 不对、watch 重启异常)，让用户扫码 6 次以上，最终才去对比 demo 源码发现是 headers 和 UIN 生成方式的差异。教训：外部协议问题第一步找参考实现逐行对比，不要让用户配合无证据的猜测反复验证。



